

1. Remix:

MessageBoard.sol

```
// SPDX-License-Identifier: MIT`
`pragma solidity ^0.8.20;`

`contract MessageBoard {`
    `struct Message {`
        `string content;`
        `address author;`
        `uint256 timestamp;`
    `}`

    ``Message[] private messages;``

    ``function postMessage(string calldata _content) external {``
        ``messages.push(Message({``
            ``content: _content,``
            ``author: msg.sender,``
            ``timestamp: block.timestamp``
        ``}));``
    ``}```

    ``function getMessageCount() external view returns (uint256) {``
        ``return messages.length;``
    ``}```

    ``function getMessage(uint256 _index) external view returns (``
        ``string memory content,``
        ``address author,``
        ``uint256 timestamp``
    ``) {``
        ``require(_index < messages.length, "Index out of range");``
        ``Message storage m = messages[_index];``
        ``return (m.content, m.author, m.timestamp);``
    ``}```
`}`
```

2. compile

3. link to Monad Testnet

1. 选择环境

2. Remix里Account显示钱包地址

4. Deploy Contract

1. 选择contract确认deploy

2. 合约地址 Contract Address

0x05832591397316F1F1d9b3598b3C079D4532BC22

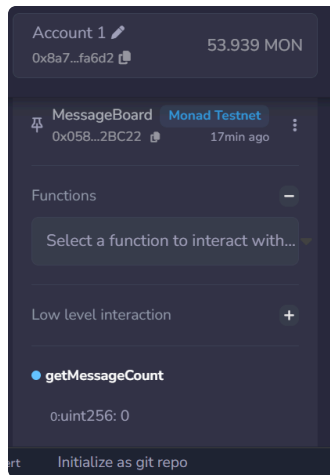
3. 部署交易 hash

0x94cad5625595faf1059d4409de17fb7f2231d8ab9fc111e9de230331b021d177

5. 调用read function

1. read function 就是不花 gas 的查询函数。

2. 在functions选择getMessageCount

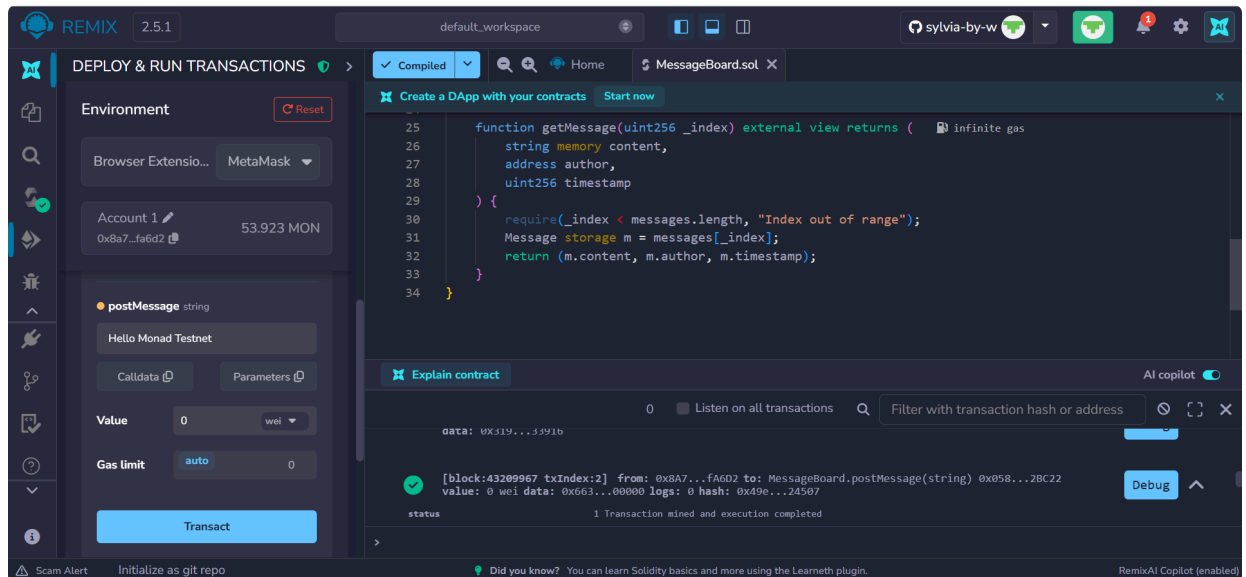


3. 如果你已经发过留言，就会返回 1 或更多。

6. 调用write function

1. write function 是会改变链上状态的函数，所以会产生交易 hash。

2. 在functions选择postMessage；输入Hello Monad Testnet确认交易

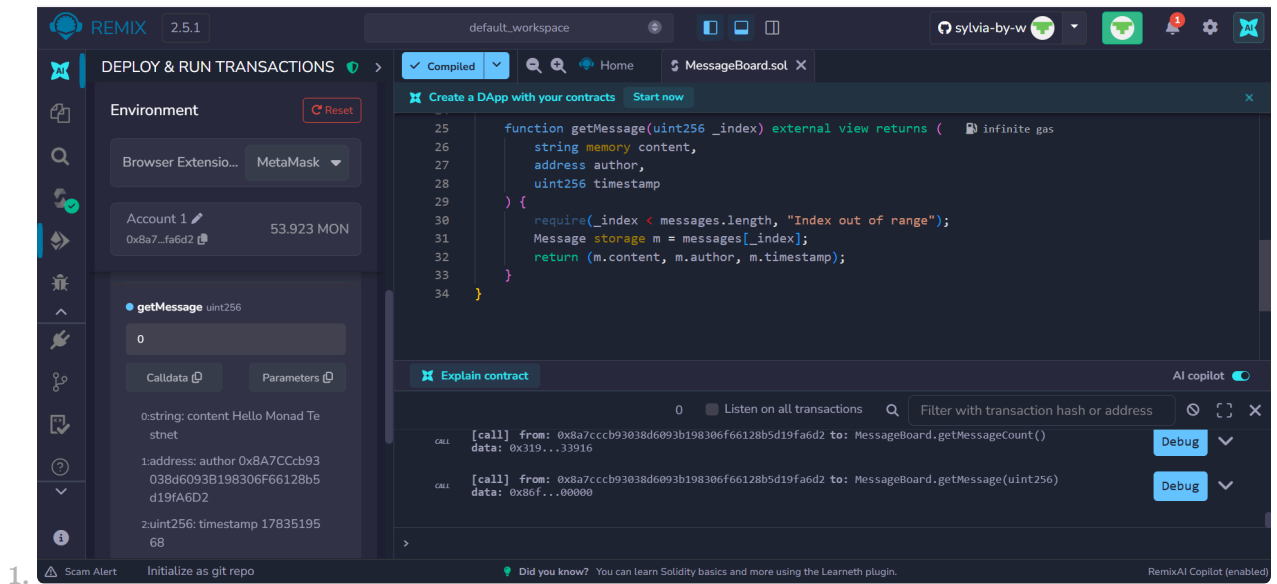


3. transaction hash：

0xc29b9105aee14597662381e1e357cd4ceb9146889291116da11057451c227530

7. 再次调用read function验证；数字会变成1

8. 再点getMessage输入o



1.

README v0.1

MessageBoard Contract v0.1

1. 合约简介

MessageBoard 是一个最小留言板智能合约，部署在 Monad Testnet 上。用户可以调用 `postMessage` 发布一条留言，合约会保存留言内容、留言者地址和发布时间。用户也可以通过 read function 查询留言总数和指定编号的留言内容。

2. 合约功能

Write Function

`postMessage(string _content)`

发布一条留言。调用后会把留言内容、调用者地址 `msg.sender` 和当前区块时间 `block.timestamp` 保存到链上。

这个函数会改变链上状态，所以需要支付测试网 gas，并会产生 transaction hash。

Read Functions

`getMessageCount()`

查询当前留言总数。这个函数只读取链上数据，不改变状态，所以不需要发起交易。

`getMessage(uint256 _index)`

根据留言编号查询某一条留言。返回内容包括：

- content: 留言内容
- author: 留言者地址
- timestamp: 留言时间

3. 部署流程

1. 在 Remix 中创建 `MessageBoard.sol` 文件。
2. 粘贴 Solidity 合约源码。
3. 使用 Solidity `^0.8.20` 编译合约。
4. 在 MetaMask 中切换到 Monad Testnet。
5. 在 Remix 的 Deploy & Run Transactions 中选择 Injected Provider - MetaMask。
6. 连接课程专用钱包。
7. 选择 `MessageBoard` 合约并点击 Deploy。
8. 在 MetaMask 中确认部署交易。
9. 保存合约地址和部署交易 hash。

4. 交互流程

1. 部署完成后，在 Remix 的 Deployed Contracts 中找到合约。
2. 调用 `getMessageCount`，确认初始留言数量。
3. 调用 `postMessage`，输入一条测试留言，例如 `Hello Monad Testnet`。
4. 在 MetaMask 中确认交易。
5. 保存这笔合约交互的 transaction hash。
6. 再次调用 `getMessageCount`，确认留言数量增加。
7. 调用 `getMessage(0)`，查看刚才发布的留言内容。

5. 本次部署信息

Contract Address:

`0x05832591397316F1F1d9b3598b3C079D4532BC22`

Deployment Transaction Hash:

0x94cad5625595faf1059d4409de17fb7f2231d8ab9fc111e9de230331b021d177

Interaction Transaction Hash:

0xc29b9105aee14597662381e1e357cd4ceb9146889291116da11057451c227530

6. 我的理解

这次任务完成了从合约源码、编译、部署到链上交互的完整流程。部署合约本身是一笔链上交易，会生成一个新的合约地址。调用 read function 只是读取链上数据，不会改变状态，也不会产生 transaction hash。调用 write function 会修改合约状态，因此需要支付 gas，并且会在区块浏览器中留下交易记录。